Hydroponics Weather Advisor Toolkit

Prompt-Powered Kickstart: Building a Hydroponics Weather Advisor with Go

Author: Ashley Kinyanjui **Institution:** Moringa School — AI Essentials for Software Developers **Date:** 19th March 2026

---

1. Title & Objective

- **Project Title:** Hydroponics Weather Advisor Toolkit
- **Technology Chosen:** Go (Golang) + OpenWeatherMap REST API
- **Why I Chose It:** I deliberately avoided Python and JavaScript to challenge myself with a statically typed, compiled language. Go is famous for its performance in backend systems and forced me to handle errors explicitly, which is a crucial skill for software developers.
- **End Goal:** Create a command-line tool for Nairobi farmers that fetches live temperature data and provides automated nutrient solution advice based on current weather conditions.

---

2. Quick Summary of the Technology

What is Go? Created by Google in 2009, Go is an open-source, compiled programming language built for speed, simplicity, and reliability. It is statically typed, meaning errors are caught before the program even runs.

Where is it used? Go powers major internet infrastructure including Docker (the world's most popular containerisation platform), Kubernetes (Google's cloud orchestration system), and Cloudflare (which handles traffic for millions of websites). It is widely used in backend APIs, DevOps tools, and cloud services.

One real-world example: Docker — used by over 20 million developers worldwide to package and ship software — is written entirely in Go. Its lightweight, fast-compiling nature makes it ideal for tools that need to start instantly and handle thousands of tasks simultaneously.

What is the OpenWeatherMap API? OpenWeatherMap is a web service that delivers live meteorological data from weather stations and satellites directly to our code in JSON format. It provides temperature, humidity, wind speed, and more for any city in the world, and offers a free tier suitable for student and personal projects.

---

3. System Requirements

Requirement	Detail
Operating System	Windows 11
Code Editor	Visual Studio Code (VS Code)
Language	Go 1.26.1
Internet Connection	Required (for live API calls)
Packages	net/http, encoding/json (Go Standard Library — no extra installs needed)
API Key	Free OpenWeatherMap account

---

4. Installation & Setup Instructions

Step 1: Install Go Downloaded the Windows installer from go.dev/dl/ and ran it. Clicked through all default settings.

Step 2: Verify Setup Ran `go version` in the VS Code terminal to confirm the PATH was correctly configured.

Expected output:

```
go version go1.26.1 windows/amd64
```

Step 3: Install the Go Extension in VS Code Searched for "Go" by the Go Team at Google in the VS Code Extensions panel and installed it. This gives syntax highlighting and error detection.

Step 4: Initialize Project Created a project folder and ran:

```
go mod init hydroponics_advisor
```

Expected output:

```
go: creating new go.mod: module hydroponics_advisor
```

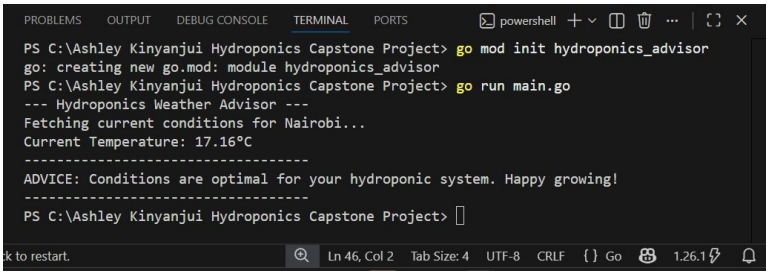

Figure 1: go mod init and go run main.go showing the program running successfully in the VS Code terminal.

Figure 1: Module initialization and first successful run of the Hydroponics Weather Advisor.

Step 5: Project Files in VS Code

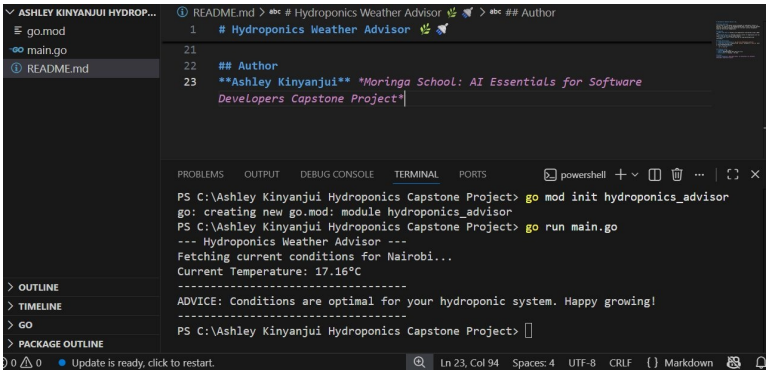

Figure 2: Project files (main.go, go.mod, README.md) visible in the VS Code Explorer panel alongside the terminal output.

Figure 2: The complete project structure in VS Code showing all three project files.

Step 6: API Access Generated a free API key at OpenWeatherMap.org by creating an account and navigating to the API Keys section of the dashboard. Note: new keys take up to 10 minutes to activate.

---

5. Minimal Working Example

This program requests live weather data for Nairobi and evaluates whether the current temperature requires action from the hydroponic farmer — specifically whether high heat requires higher oxygenation for plant roots, or whether cold temperatures may slow nutrient uptake.

Code (main.go):

```go
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

func main() {
    // Replace with your own OpenWeatherMap API key
    apiKey := "YOUR_API_KEY_HERE"
    city := "Nairobi"
    url := fmt.Sprintf(
        "http://api.openweathermap.org/data/2.5/weather?
q=%s&appid=%s&units=metric",
        city, apiKey,
    )

    fmt.Println("--- Hydroponics Weather Advisor ---")
    fmt.Printf("Fetching current conditions for %s...\n", city)

    // Making the live API call
    resp, err := http.Get(url)
    if err != nil {
        fmt.Println("Error: Could not reach the weather service.")
        return
    }
```

```go
            defer resp.Body.Close()

            // Parsing the JSON response
            var data map[string]interface{}
            json.NewDecoder(resp.Body).Decode(&data)

            // Navigating the nested JSON to extract temperature
            mainData := data["main"].(map[string]interface{})
            temp := mainData["temp"].(float64)

            fmt.Printf("Current Temperature: %.2f°C\n", temp)

            // Hydroponic advice logic based on temperature
            fmt.Println("-----------------------------------")
            if temp > 28 {
                fmt.Println("ADVICE: High heat! Increase oxygenation in your
reservoir.")
            } else if temp < 15 {
                fmt.Println("ADVICE: Low temp! Monitor for nutrient
lockout.")
            } else {
                fmt.Println("ADVICE: Conditions are optimal. Happy
growing!")
            }
            fmt.Println("-----------------------------------")
        }
```

How to run:

```
go run main.go
```

Expected Output:

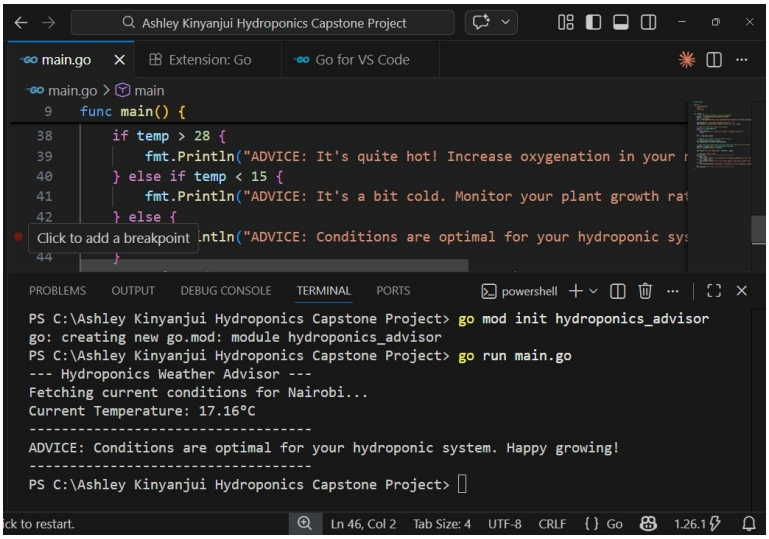

Figure 3: The full main.go code and the successful terminal output side by side in VS Code.

Figure 3: Complete view of main.go and the successful execution output showing the live temperature advice.

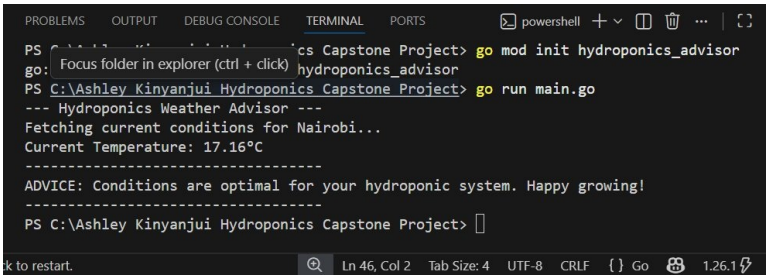

Figure 4: Close-up of the terminal showing the Hydroponics Weather Advisor live output.

Figure 4: Live terminal output — Current Temperature: 17.16°C with optimal conditions advice.

---

6. AI Prompt Journal

Prompt 1: > "I am a complete beginner. Help me install Go on Windows and fix the 'term not recognized' error without using Python or JavaScript."

AI Response Summary: The AI guided me through the Go installation process on Windows, explaining how to download the MSI installer, run it, and then restart VS Code to refresh the system PATH. It also explained what `go mod init` does and why it is needed before writing any code.

Evaluation: Extremely helpful. What would have taken hours of reading documentation was resolved in minutes. The step-by-step format was perfect for a beginner.

Prompt 2: > "How do I make an HTTP GET request in Go and parse a JSON response? I want to fetch weather data from OpenWeatherMap and extract the temperature field."

AI Response Summary: The AI explained the `net/http` and `encoding/json` packages, showed how to use `http.Get()`, and taught me how to navigate nested JSON using `map[string]interface{}` and type assertions like `.(float64)`.

Evaluation: Very helpful. JSON parsing in Go is more complex than in Python, and the AI broke it down into simple steps with clear examples.

Prompt 3: > "My Go program compiles but crashes with: interface conversion: interface {} is nil, not map[string]interface{}. What does this mean and how do I fix it?"

AI Response Summary: The AI identified that this error usually means the API returned an error response (like `{"cod":401,"message":"Invalid API key"}`) instead of weather data. It advised printing the raw response to diagnose, then checking that the API key was active.

Evaluation: Extremely helpful. This was my most frustrating error. The AI pinpointed the cause immediately — the key needed 10 minutes to activate after account creation.

Prompt 4: > "How do I write a README.md for a Go project on GitHub? What sections should I include for a beginner capstone project?"

AI Response Summary: The AI suggested sections including Project Overview, Features, Setup Instructions, Technical Stack, and Author. It also provided Markdown formatting tips.

Evaluation: Helpful for structuring professional documentation quickly and confidently.

Overall Reflection: Using generative AI as a learning tool dramatically accelerated my progress. Tasks that would have taken hours of reading documentation — like understanding Go's JSON parsing or diagnosing a nil interface error — were resolved in minutes. The key skill I developed was learning to describe my problem clearly and specifically. Vague prompts gave vague answers; detailed, specific prompts gave precise, usable solutions. AI served as a "Pair Programmer" that allowed me to move significantly faster while still understanding every line of code I wrote.

---

7. Common Issues & Fixes

Error 1: `go: command not found`

What happened: After installing Go, the terminal did not recognise the `go` command.

Fix: Closed VS Code completely and reopened it. The terminal needs to reload environment variables after a new installation. Ran `go version` again to confirm.

Error 2: API Key Not Working (401 Unauthorized)

What happened: The program compiled and ran but crashed with a nil interface error.

Root cause: The OpenWeatherMap API key was not yet active. New accounts receive a key immediately but it takes up to 10 minutes to become functional.

Fix: Waited 10 minutes, then tried again. To diagnose, added `fmt.Println(data)` after decoding to print the raw API response. Saw `map[cod:401 message:Invalid API key]` which confirmed the key was inactive.

Error 3: `interface conversion: interface {} is nil`

What happened: Program crashed when trying to extract temperature.

Fix: This happens when the JSON response does not contain the expected `"main"` field — usually because the API returned an error. Fixed by resolving the API key issue above first.

Error 4: `go mod init` gives error about existing module

What happened: Running `go mod init` twice in the same folder caused an error.

Fix: The `go.mod` file only needs to be created once. If it already exists, skip this step.

Error 5: API Key Exposed on GitHub (GitGuardian Security Alert)

What happened: After pushing code to GitHub, I received an email from GitGuardian warning that my OpenWeatherMap API key was exposed publicly in my repository. The alert looked like this:

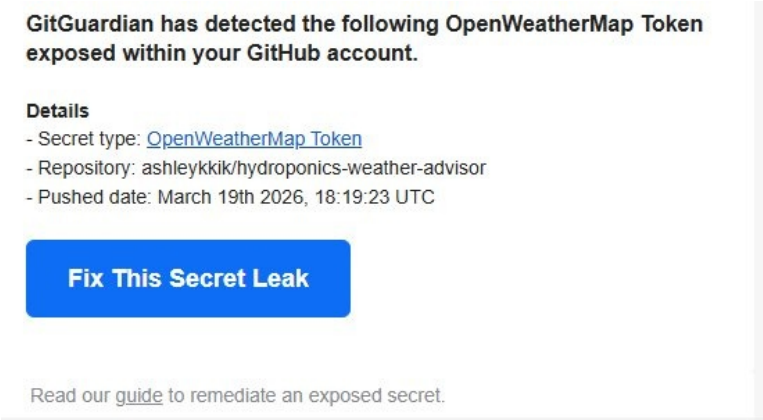

Figure 5: GitGuardian security alert email warning about the exposed OpenWeatherMap API key.

Why this is serious: When an API key is pushed to a public GitHub repo, anyone in the world can see it, use it, and rack up charges or abuse on your account. GitGuardian automatically scans public repos and alerts developers when this happens.

Fix — Two steps required:

Step 1 — Revoke the exposed key immediately: - Log into OpenWeatherMap.org - Go to API Keys in your dashboard - Delete the exposed key and generate a brand new one - Save the new key somewhere private (e.g. your phone notes — never in your code)

Step 2 — Remove the key from your code and push the fix: - In main.go, replace the real key with a placeholder:

```
        apiKey := "YOUR_API_KEY_HERE"
```

- Then push the fix to GitHub:

```
git add .
git commit -m "Remove exposed API key"
git push
```

Lesson learned: Never commit real API keys, passwords, or secrets to GitHub. For future projects, store sensitive keys in environment variables or a `.env` file, and add `.env` to your `.gitignore` file so it never gets pushed.

Helpful resources used during debugging:

- Go official documentation: https://pkg.go.dev/net/http
- OpenWeatherMap API docs: https://openweathermap.org/current
- Stack Overflow: searched “golang json decode nil interface”

---

8. References

Official Documentation:

- Go Official Website: https://go.dev
- Go Downloads: https://go.dev/dl/
- Go Tour (Interactive Beginner Guide): https://go.dev/tour/
- Go net/http Package: https://pkg.go.dev/net/http
- Go encoding/json Package: https://pkg.go.dev/encoding/json
- OpenWeatherMap API Docs: https://openweathermap.org/current

Video Tutorials:

- Go in 100 Seconds (Fireship): https://www.youtube.com/watch?v=446E-r0rXHI
- Go Programming — Full Beginner's Course (freeCodeCamp): https://www.youtube.com/watch?v=un6ZyFkqFKo
- OpenWeatherMap API Tutorial: https://www.youtube.com/watch?v=9P5MY_2i7K8

Helpful Blog Posts & Forums:

- How to Parse JSON in Go: https://gobyexample.com/json
- Go HTTP Requests Guide: https://gobyexample.com/http-clients
- Stack Overflow — Go JSON parsing: https://stackoverflow.com/questions/tagged/go+json
- Getting Started with OpenWeatherMap API: https://openweathermap.org/appid

---

Submitted as part of the Moringa School AI Essentials for Software Developers Capstone, 2026. All code written and tested on Windows 11, Go 1.26.1, VS Code. GitHub Repository: https://github.com/ashleykkik/hydroponics-weather-advisor